

Week 9 Summer Homework: Graph Signal Processing for Smart Grids

Goal: connect graph topology, node measurements, the graph Laplacian, smoothing, and anomaly detection.

AI policy for this homework. You may use AI or web search to learn Python syntax, understand documentation, debug error messages, and clarify concepts. You may **not** ask AI to write the required core code for you. Your submitted scripts must be your own manual coding work. If you use AI for explanation, write one sentence describing what it helped you understand.

Mathematical reminder. A graph signal is one number attached to each node of a graph. For an unweighted graph, the adjacency matrix A stores which nodes are connected, and the degree matrix D stores how many neighbors each node has. The graph Laplacian is

$$L = D - A.$$

When L acts on a graph signal x , the result Lx measures how much each node disagrees with its neighbors. Low graph frequency means smooth changes across connected nodes; high graph frequency means sharp changes across connected nodes.

Monday: Draw a Mini Power-Grid Graph

Design a small power-grid graph with 6–8 nodes. Your nodes may represent a generator, substations, buses, neighborhoods, or sensors. Your edges should represent transmission or distribution connections.

1. Draw the graph by hand or with Python.
2. Label every node with a short physical meaning, such as **Generator**, **Substation A**, **Bus 1**, or **Sensor 3**.
3. Write the edge list for your graph, such as $(0,1)$, $(1,2)$, $(1,3)$.
4. In three or four sentences, explain why your graph is a reasonable simplified model of a power network.

Tuesday: Build A , D , and L Manually

Using the graph from Monday, write a Python script that builds the adjacency matrix A , the degree matrix D , and the Laplacian matrix $L = D - A$.

1. Create an all-zero adjacency matrix with `numpy`.
2. Use your edge list and a `for` loop to fill in A .
3. Compute each node degree manually by summing rows of A .
4. Build D as a diagonal matrix.
5. Compute $L = D - A$ and print all three matrices clearly.

Manual coding rule. Do not use NetworkX or PyGSP to build A , D , or L for Tuesday. You may use `numpy.zeros`, array indexing, loops, and basic arithmetic. The goal is to understand the matrix logic yourself.

Wednesday: Create a Graph Signal and Find Disagreement

Create a graph signal representing one smart-grid measurement at every node. Choose one of these examples: voltage magnitude, frequency deviation, load, current, or temperature.

1. Create a vector $\mathbf{x}$ with one value for each node.
2. Make most neighboring values similar, but intentionally create one suspicious node value.
3. Compute `disagreement = L @ x`.
4. Print each node's value and disagreement score.
5. Identify the most suspicious node and explain your reasoning in three sentences.

Hint: A large positive or negative disagreement score means the node is very different from its graph neighbors. Use absolute value if you want to rank how unusual each node is.

Thursday: PyGSP Visualization and Simple Smoothing

Use PyGSP to visualize your graph signal and apply a simple smoothing operation.

1. Build your graph in PyGSP using your adjacency matrix A .
2. Assign 2D coordinates so the graph layout is easy to read.
3. Plot your original graph signal with a colorbar.
4. Apply one simple smoothing step: $\mathbf{x}_{\text{smooth}} = \mathbf{x} - \alpha * (\mathbf{L} @ \mathbf{x})$ with a small α , such as 0.05 or 0.10.
5. Plot the smoothed signal and compare it with the original signal.
6. Write four or five sentences: What changed after smoothing? Did smoothing help reveal noise, or could it hide a real fault?

Important: If PyGSP is unavailable on your computer, still complete the smoothing calculation with `numpy` and make a simple node-color plot using any plotting tool you already know. Clearly state which tool you used.

Friday: Feynman Technique Post

Make a beginner-friendly post explaining Graph Signal Processing using a power-grid example.

1. Explain what a graph signal is without using advanced math.
2. Explain the graph Laplacian as a neighbor-comparison tool.
3. Explain graph frequency by comparing it with FFT: FFT finds slow and fast changes over time; GSP finds smooth and sharp changes over a network.
4. Include one small drawing, diagram, or screenshot from your work.
5. End with one real smart-grid application, such as abnormal sensor detection, smoothing noisy measurements, filling missing node values, or understanding how signals spread.

Submission checklist: Monday graph drawing and edge list; Tuesday A , D , and L script; Wednesday graph signal and disagreement analysis; Thursday PyGSP or Python visualization with smoothing; Friday Feynman note.